

DSLS Programming 5 Lecture 2: OpenMPI

1. Prelim

```
(setq org-babel-python-command "/usr/bin/python3")
```

2. About OpenMPI

- We discussed in the first lecture about SLURM that we needed two properties for libraries;
 1. to manage resources on a cluster fairly and start programs with them
 2. to get around the security restrictions on programs sharing memory

3. Model: Single Program, Multiple Data

- In distributed computing theory, two paradigms are distinguished;
 1. Single Program, Multiple Data (SPMD)
 2. Multiple Program, Multiple Data (MPMD)
- In this lesson, we deal with SPMD; OpenMPI is ideal for this

4. Upsides of MPI & mpi4pi

- MPI ("Message Passing Interface") is an industry standard library to solve communication and synchronization across clusters
- If you were wondering; a. "How do I know how many processes there are?" b. "How do I send data to another node in the cluster?" c. "How do I know when all nodes are done with their calculation?"
- MPI is the answer!
- The other good thing about it is, together with SLURM, it provides a way to start programs on multiple nodes

5. Downsides of MPI & mpi4pi

- OpenMPI is still fairly low level; i.e. it provides communication channels but you have to divide data over them
- You also have to explicitly send and receive data
- check that all parts of your calculation are done, and combine the result
- There is no automatic error handling or save/backup of data in crashes

6. Basics: setting up

- You start every MPI program by setting up the "world" (talk about ambitious!)

```
from mpi4py import MPI  
comm = MPI.COMM_WORLD  
size = comm.Get_size()
```

- what this does is set up a communications channel to the "world"

- but what is the "world" ?
- It is all the copies of your program, started on all nodes you requested from the cluster!

7. Starting MPI programs

- You can start MPI programs "by hand" using the mpiexec command line program.

```
mpiexec -np 16 python3 my_mpi_program.py
```

- this runs the python3 interpreter on your my_mpi_program.py script, with 16 separate copies
- The copies are separate python3 interpreters which don't share data!
- so, you need mpi4py to communicate between them
- but first, we need to know "who we are", i.e. which copy of the program we are
- in MPI, this is your "rank"

```
if __name__ == "__main__":
    # we're running as a separate process
    # Check which rank we are
    rank = comm.Get_rank()
    print("Rank: ", rank)
```

- There is always one controller process, called rank 0
- It is responsible for;
 - Getting the data ready (load file)
 - Setting up appropriate communications to the other process
 - Dividing up and sending the data to the other processes
 - Getting the result and checking all processes are done

```
if rank == 0:
    # Controller process
    print("Controller process")
else:
    # Workers process
    print(f"Worker process {rank}")
```

- This is another example of a guard pattern (like the module guard)
- In most MPI programs you only want to distinguish between rank 0 and other rank (> 0)
- But ranks are useful in directly addressing programs!

8. Communication

- The most important task of MPI is passing messages :-)
- There are two "Styles" of communication; synchronous, or asynchronous
- Let's start with synchronous: that way the sender (e.g. rank 1) knows the receiver (e.g. rank 1) got it

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
if rank == 0:
    # Controller process
    print("Controller process")
```

```

comm.send("Hello from rank 0", dest=1)
elif rank == 1:
    # Worker nr 1 process
    print("Worker process 1")
    data = comm.recv(source=0)
    print(f"Rank 1 Received: {data}")
else:
    # Some other worker process
    print(f"Worker process {rank}")

```

```

Worker process 2
Controller process
Worker process 3
Worker process 1
Rank 1 Received: Hello from rank 0

```

- what's happening here is that one worker gets data from the controller, synchronously (it waits for it)
- Notice the order changes (everytime; stochastic!)

9. Datatypes

- the data can be any python object!
- it has to be "serializable" though by the pickle module
- In practice this means it has to be a pure python (or numpy) object
- You can also use "Upper-case" methods (e.g. `comm.Send` and `comm.Recv`) for explicit typing

```
comm.Send([np.random.randint(100,1), MPI.INT], dest=10, tag=11)
```

- Notice the capital "S" in `comm.Send` !

10. Broadcasting

- Usually, you don't want to go through the trouble of sending data one-by-one
- Although sometimes for efficiency this can be good!
- You can send data to all other ranks at once
- You can either send all data in its entirety to all workers, or distribute it
- Or, you can send only part of the data to each rank, managed by the scatter/gather functions

```

(setq org-babel-python-command "/usr/bin/python3")
(print org-babel-python-command)

```

```

from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if rank == 0:
    # Controller process
    print("In rank 0")
    # Make some data
    data = {1 : 'a', 2 : 'b', 3 : 'c'}
    print("Rank 0 sends data!")
    data = comm.bcast(data, root=0)
else:
    # We're a worker process

```

```
print(f"In Worker process {rank}")
data = comm.bcast(None, root=0)
print(f"Worker process {rank} picks data '{data[rank]}' to UPPER: '{data[rank].upper}'")
```

```
In Worker process 1
In Worker process 3
In rank 0
Rank 0 sends data!
Worker process 1 picks data 'a' to UPPER: 'A'
Worker process 3 picks data 'c' to UPPER: 'C'
In Worker process 2
Worker process 2 picks data 'b' to UPPER: 'B'
```

11

- How do you get the results back from the workers? You can:
 - Send it one by one
 - Send it in one go using a broadcast
 - Use a smarter algorithm that reduces (combines) results for less I/O

12. Send it one by one

- This is a good opportunity for asynchronous communication, since;
 - The controller (rank 0) doesn't know when or in what order workers are done
 - This is called a "polling" or "event-loop" pattern
 - It is especially useful when the number of results is unknown

13. Async return

```
if rank == 0:
    data = { i : randint(3, 10000) for i in range(1,size)}
    comm.bcast(data, root=0) # this is synchronous
    result_reqs = [ comm.irecv(source=i) for i in range(1,size)]
    results = []
    while True: # event loop
        for req in result_reqs: # check on prime calculation
            result = req.test() # NB test returns results!
            if result[0]:
                results.append(result[1])
                result_reqs.remove(req)
        if not result_reqs: break # we've handled all requests
        # do other stuff here
    print(f"Rank 0 got all results: {results}")
else:
    data = comm.bcast(None, root=0)
    result = { data[rank] : is_prime(data[rank])}
    req_result = comm.isend(result, dest=0)
    req_result.wait() # sync; but we could do other stuff now
```

14. Output

```
Rank 0 got all results: [{7858: False}, {5111: False}, {3073: False}]
```

15. Scatter / Gather

- To solve the problem with broadcast sending **all** the data you can use **scatter** and its reverse **gather**
- Notice how each worker rank only sees singular data but rank 0 sees lists
- This reduces network traffic and space

16. Scatter / Gather source

```
if rank == 0:
    data = [ randint(3, 10000) for i in range(1,size+1)]
    comm.scatter(data, root=0) # this is synchronous
    results = comm.gather(None, root=0)
    print(f"Results: {results}")
else:
    data = comm.scatter([], root=0)
    print(f"rank {rank} got {data} to work on")
    prime = is_prime(data)
    comm.gather((data, prime), root=0)
```

```
Hello from assemblix2019 rank 1 of 5
Hello from assemblix2019 rank 2 of 5
Hello from assemblix2019 rank 3 of 5
Hello from assemblix2019 rank 0 of 5
rank 3 got 625 to work on
rank 1 got 3442 to work on
rank 2 got 7830 to work on
Hello from assemblix2019 rank 4 of 5
rank 4 got 5277 to work on
Results: [None, (3442, False), (7830, False), (625, False), (5277, False)]
```

17. Results

```
rank 4 got 6338 to work on
rank 1 got 4013 to work on
rank 2 got 2582 to work on
rank 3 got 1992 to work on
Results: [None, (4013, True), (2582, False), (1992, False), (6338, False)]
```

18. MapReduce

- There are also primitives which make programming even easier; one of these is the **Mapreduce** function
- This will be further introduced in the context of the Spark system
- For now, we will focus on explicitly handling the data in the controller/worker rank

19. OpenMPI and SLURM

- SLURM has direct support for OpenMPI; if you use **srun** in your batch scripts it works!

```
#!/bin/bash
#SBATCH --ntasks=16
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=8 #←not required, just handy
#SBATCH --cpus-per-task=1 #←not required, just handy
srun python3 scatter_gather.py
```

20. Performance

- In this course, we will keep an eye on the performance of our distributed programs
- There is a cost (overhead) to distributing data and computations which should not exceed the gain!
- Many facilities for gathering program time information exist; we will cover both general and framework-specific ones
- The first utility to test is the Unix `/usr/bin/time` command; it is integrated into the next assignment.
- It works at the program (shell) level, so gives "overall" information without considering parts of your program!

Author: Martijn Herber

Created: 2025-09-18 do 15:30

[Validate](#)

Collective operations are functions in the Message Passing Interface (MPI) that involve all processes within a given communicator (like `MPI.COMM_WORLD`). Every rank must call the same function for the operation to succeed, and they typically used for moving data efficiently across the entire group or creating synchronization points.

OPERATIONS: Bcast, Scatter, Gather

Data Distribution (One-to-All)

These operations take data from a single root process and send it to all other processes.

Bcast (Broadcast): Sends the same, identical data object from the root to all other ranks.

Scatter: Divides a single large array or list held by the root into unique chunks, sending one chunk to each rank.

Data Collection & Combination (All-to-One / All-to-All)

These operations collect data from all ranks and either combine it on the root or distribute the combined result to every

Gather: Collects unique data chunks from all ranks and combines them into one ordered array/list on the root process.

Collective means all workers (0 and others)
MUST
run it.

A	B	C	D	E
Method	Type of Operation	Communication Pattern	Blocking Nature	Primary Purpose
comm.bcast()	Collective	One-to-All	Blocking	Sends the same, identical data from the root to all ranks.
comm.scatter()	Collective	One-to-All	Blocking	Divides a list/array into unique chunks and sends one to each rank.
comm.gather()	Collective	All-to-One	Blocking	Collects unique data from all ranks and combines it on the root (Rank 0).
comm.send()	Point-to-Point	One-to-One	Blocking	Sends data and stops until the transfer is initiated/confirmed by MPI.
comm.recv()	Point-to-Point	One-to-One	Blocking	Stops and waits until the message (matching source/tag) has fully arrived.
comm.isend()	Point-to-Point	One-to-One	Non-Blocking	Initiates the send, returns immediately, and requires later synchronization (.wait()) for memory safety.
comm.irecv()	Point-to-Point	One-to-One	Non-Blocking	Initiates the receive, returns immediately with a Request object for later checking (.test()).
req.wait()	Synchronization	N/A	Blocking	Forces the process to stop and wait until a non-blocking operation (isend or irecv) is 100% complete.
req.test()	Synchronization	N/A	Non-Blocking	Checks the status of a non-blocking request, returning a flag without stopping the code flow.